AgriMind — Full project workflow (Google Colab)

For submission: use **Runtime → Run all**, wait until every cell finishes, then **File → Download → Download .ipynb** so your teacher sees **code and outputs** together.

This notebook mirrors the local project:

- Real APY crop data + state climate merge
- Preprocessing and feature engineering
- Yield models (Linear, HGB, Tree, RF, XGB) with **metrics table**
- Full `run_all` pipeline (artifacts + crop classifier + SHAP plot)
- Sample prediction

Repo folder on Colab: usually `/content/ml-project-main/backend` (GitHub ZIP) or `/content/MLproject/backend` (git clone).

1) Install packages (same stack as backend ML pipeline)

```
%%capture
!pip install -q numpy pandas scikit-learn xgboost joblib shap matpl
```

2) Point Colab at your **backend** folder

After a **new runtime**, `/content` is empty except `sample_data` — you must either:

- **Set `GITHUB_USER` + `REPO_NAME`** below and let the cell **download + unzip** your repo, or
- Upload your project ZIP to `/content` and unzip (then re-run this cell).

The cell **searches all of `/content`** for a folder that contains both `app.py` and `ml_models/`.

```
%matplotlib inline
import os, sys
import urllib.request
import zipfile
```

```
from pathlib import Path

# If backend not found, download public GitHub repo as ZIP (edit th
GITHUB_USER = "pranjali999"  # e.g. "your-github-username"
REPO_NAME = "ml-project"    # e.g. "ml-project"
BRANCH = "main"


def find_backend():
    flat = [
        Path("/content/ml-project-main/backend"),
        Path("/content/MLproject/backend"),
        Path("/content/MLproject-main/backend"),
        Path("/content/backend"),
        Path.cwd(),
    ]
    for c in flat:
        try:
            if c.is_dir() and (c / "ml_models").is_dir() and (c / "
                return c.resolve()
        except OSError:
            pass
    root = Path("/content")
    if root.is_dir():
        for child in sorted(root.iterdir()):
            if not child.is_dir() or child.name.startswith("."):
                continue
            for sub in (child / "backend", child):
                if (
                    sub.is_dir()
                    and (sub / "ml_models").is_dir()
                    and (sub / "app.py").is_file()
                ):
                    return sub.resolve()
        for app in root.rglob("app.py"):
            d = app.parent
            if (d / "ml_models").is_dir():
                return d.resolve()
    return None


BACKEND = find_backend()

if BACKEND is None and GITHUB_USER and REPO_NAME:
    root = Path("/content")
    zip_path = root / "repo_source.zip"
    url = f"https://github.com/{GITHUB_USER}/{REPO_NAME}/archive/re
    print("Downloading:", url)
```

```
        urllib.request.urlretrieve(url, zip_path)
        with zipfile.ZipFile(zip_path, "r") as z:
            z.extractall(root)
        BACKEND = find_backend()

    if BACKEND is None:
        raise FileNotFoundError(
            "No backend found (need app.py + ml_models/).\n"
            "Folders in /content: " + str([p.name for p in Path('/conte
            "Fix: set GITHUB_USER and REPO_NAME in this cell (public re
        )

    os.chdir(BACKEND)
    sys.path.insert(0, str(BACKEND.resolve()))
    os.environ["MPLBACKEND"] = "inline"
    print("BACKEND:", BACKEND.resolve())
```

```
Downloading: https://github.com/pranjali999/ml-project/archive/refs/
BACKEND: /content/ml-project-main/backend
```

3) Download crop production CSV (if missing)

```
import pandas as pd
from pathlib import Path
from IPython.display import display

DATA_DIR = Path("data")
DATA_DIR.mkdir(parents=True, exist_ok=True)
csv_path = DATA_DIR / "crop_production_india.csv"
URL = "https://raw.githubusercontent.com/vish-manit/Crop-Production
MAX_ROWS = 50000

if not csv_path.is_file():
    df_dl = pd.read_csv(URL, low_memory=False)
    if MAX_ROWS and len(df_dl) > MAX_ROWS:
        df_dl = df_dl.sample(n=MAX_ROWS, random_state=42).reset_ind
    df_dl.to_csv(csv_path, index=False)
    print("Saved rows:", len(df_dl))
else:
    df_dl = pd.read_csv(csv_path, nrows=5)
    print("CSV already present; showing 5-row peek:")
    display(df_dl)
```

```
Saved rows: 50000
```

4) Load pipeline data + **preview** (outputs for report)

```
from IPython.display import display
from ml_models.pipeline.data_loader import load_dataset
from ml_models.pipeline.preprocess import clean_raw_frame
from ml_models.pipeline.features import add_yield_and_rainfall_cate

raw, src = load_dataset(verbose=True)
print("Source:", src, "| Raw rows:", len(raw))
clean = clean_raw_frame(raw)
print("After clean:", len(clean))
feat = add_yield_and_rainfall_category(clean)
print("After yield filter:", len(feat))
print("Yield mean / std:", round(feat["yield"].mean(), 3), round(fe
display(feat.head(10))
display(feat[["yield", "rainfall", "temperature", "area"]].describe
```

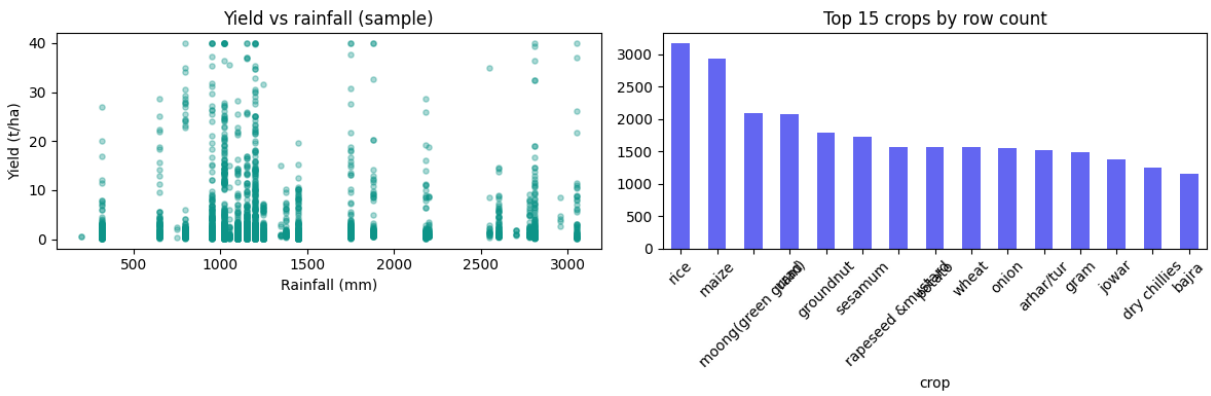
```
Added rainfall from state climate normals (annual mm; see data/india
Added temperature from state climate normals (mean annual °C; same f
Source: file:/content/ml-project-main/backend/data/crop_production_i
After clean: 49302
After yield filter: 47050
Yield mean / std: 2.952 5.694
```

	state	district	crop	year	season	area	production	rain
0	nagaland	phek	tapioca	2017	kharif	180.0	3650.0	
1	andhra pradesh	rangareddi	banana	2005	whole year	40.0	836.0	
2	assam	jorhat	sweet potato	2011	whole year	74.0	244.0	
3	jammu and kashmir	udhampur	rapeseed &mustard	2006	rabi	2833.0	2079.0	
4	nagaland	phek	tapioca	2019	kharif	200.0	4052.0	
5	rajasthan	banswara	masoor	2008	rabi	5.0	4.0	
6	west bengal	hooghly	peas & beans (pulses)	2004	rabi	554.0	414.0	
7	uttar pradesh	amethi	coriander	2012	rabi	29.0	15.0	
8	west bengal	purulia	linseed	2011	rabi	417.0	80.0	
9	uttarakhand	almora	barley	2014	rabi	2692.0	2778.0	

	yield	rainfall	temperature	area
count	47050.000000	47050.000000	47050.000000	4.705000e+04
mean	2.952254	1317.864187	25.598806	1.162561e+04
std	5.693972	617.429037	2.105755	4.038598e+04
min	0.108073	200.000000	5.500000	8.000000e-01

5) **EDA charts** (inline — visible in saved .ipynb)

```
import matplotlib.pyplot as plt
import numpy as np

s = feat.sample(min(4000, len(feat)), random_state=42)
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].scatter(s["rainfall"], s["yield"], alpha=0.35, s=14, c="#0d
axes[0].set_xlabel("Rainfall (mm)")
axes[0].set_ylabel("Yield (t/ha)")
axes[0].set_title("Yield vs rainfall (sample)")
top = feat["crop"].value_counts().head(15)
top.plot(kind="bar", ax=axes[1], color="#6366f1")
axes[1].set_title("Top 15 crops by row count")
axes[1].tick_params(axis="x", rotation=45)
plt.tight_layout()
plt.show()
```



6) **Model comparison table** (R^2, RMSE, MAE)

Uses a random subsample so this cell finishes in reasonable time; same estimators as `train_regressors.py`.

```
from ml_models.pipeline.train_regressors import train_and_compare

DEMO_N = min(20000, len(feat))
demo_df = feat.sample(n=DEMO_N, random_state=42).reset_index(drop=T
best_pipe_demo, table, meta = train_and_compare(demo_df)
display(table.round(4))
print("Best model (by test R²):", meta["best_model_name"])
```

model	r2	rmse	mae
LinearRegression	0.6089	3.2855	1.5667
HistGradientBoosting	0.8378	2.1160	0.9108
DecisionTreeRegressor	0.7952	2.3777	1.1673
RandomForestRegressor	0.8289	2.1733	0.9862
XGBRegressor	0.8366	2.1235	0.8724

```
Best model (by test R²): HistGradientBoosting
```

7) **Full pipeline** — same as local `python -m ml_models.pipeline.run_all`

Writes `yield_xgb.joblib`, crop model, EDA PNGs, SHAP summary. **Scroll this output** for crop classifier accuracy and full logs.

```
import os
os.environ["AGRIMIND_MAX_TRAIN_ROWS"] = "50000"
os.environ.pop("AGRIMIND_USE_SYNTHETIC_DATA", None)

from ml_models.pipeline.run_all import main
main()
```

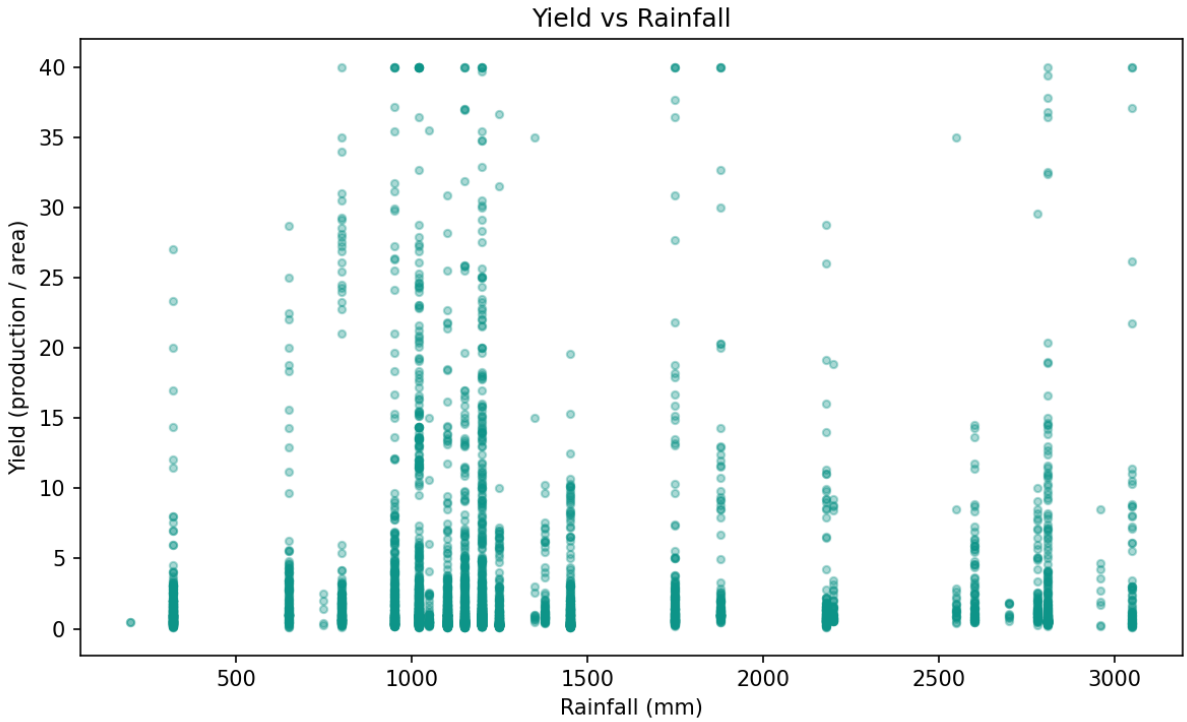
```
           coriander       0.00      0.00      0.00       133
        cotton(lint)       0.00      0.00      0.00       198
      cowpea(lobia)       0.00      0.00      0.00        49
       dry chillies       0.00      0.00      0.00       250
             garlic       0.08      0.10      0.09       146
             ginger       0.06      0.07      0.06       136
               gram       0.00      0.00      0.00       297
          groundnut       0.04      0.13      0.07       358
          guar seed       0.00      0.00      0.00        56
         horse-gram       0.00      0.00      0.00       146
```

```
                  jowar       0.00      0.00      0.00       275
                   jute       0.00      0.00      0.00        56
                khesari       0.00      0.00      0.00        53
                linseed       0.00      0.00      0.00       174
                  maize       0.08      0.53      0.13       588
                 masoor       0.00      0.00      0.00       186
                  mesta       0.00      0.00      0.00        72
     moong(green gram)       0.00      0.00      0.00       419
                   moth       0.00      0.00      0.00        35
             niger seed       0.00      0.00      0.00        83
         oilseeds total       0.00      0.00      0.00        13
                  onion       0.00      0.00      0.00       309
    other  rabi pulses       0.00      0.00      0.00       139
          other cereals       0.00      0.00      0.00        37
   other kharif pulses       0.00      0.00      0.00       154
        other oilseeds       0.00      0.00      0.00        35
   other summer pulses       0.00      0.00      0.00         2
 peas & beans (pulses)       0.07      0.05      0.06       203
                 potato       0.05      0.04      0.04       314
                   ragi       0.00      0.00      0.00       150
     rapeseed &mustard       0.10      0.04      0.06       315
                   rice       0.11      0.39      0.17       634
             safflower       0.00      0.00      0.00        52
              sannhamp       0.00      0.00      0.00        71
               sesamum       0.00      0.00      0.00       344
          small millets       0.00      0.00      0.00       194
              soyabean       0.00      0.00      0.00       145
             sugarcane       0.00      0.00      0.00       103
             sunflower       0.00      0.00      0.00       189
           sweet potato       0.00      0.00      0.00       159
               tapioca       0.00      0.00      0.00        61
               tobacco       0.00      0.00      0.00       106
              turmeric       0.00      0.00      0.00       164
                  urad       0.05      0.07      0.06       415
                 wheat       0.05      0.03      0.04       312

              accuracy                           0.08      9410
             macro avg       0.02      0.03      0.02      9410
          weighted avg       0.03      0.08      0.04      9410

Saved /content/ml-project-main/backend/ml_models/artifacts/crop_mo

=== 7. SHAP (XAI) ===
Saved SHAP summary -> /content/ml-project-main/backend/ml_models/a

=== 8. Optional Prophet (forecast artifact) ===
Saved prophet_yield.joblib
```

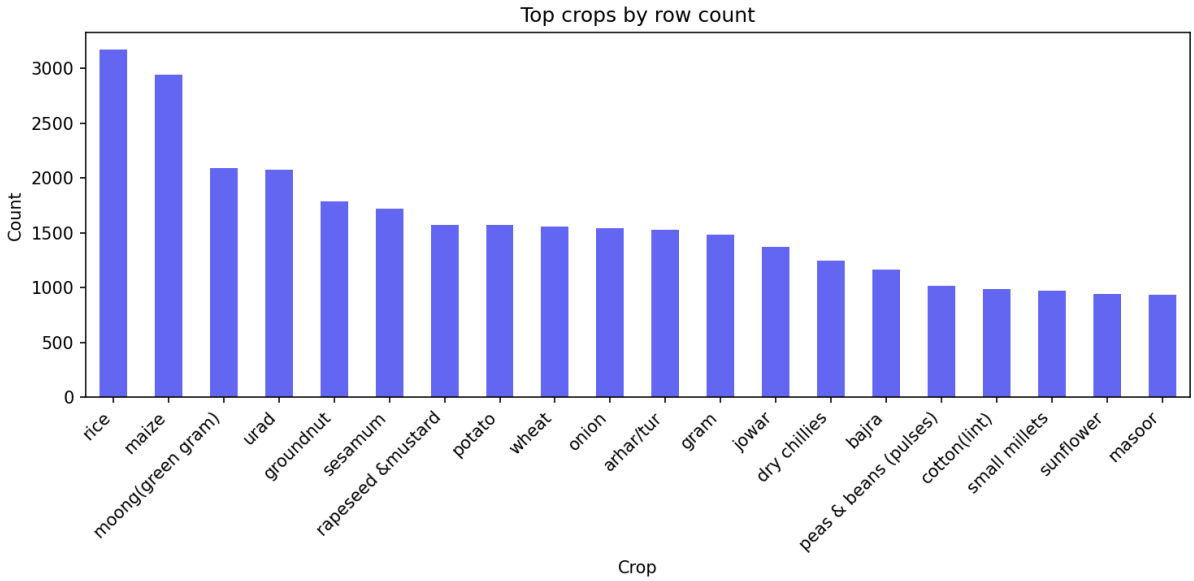
8) **Saved figures** from disk (EDA + SHAP)

```
from IPython.display import Image, display
from pathlib import Path

eda = Path("ml_models/artifacts/eda")
for name in ["yield_vs_rainfall.png", "crop_distribution.png", "cor
    p = eda / name
    if p.is_file():
        print(name)
        display(Image(filename=str(p)))
    else:
        print("(missing)", p)
```

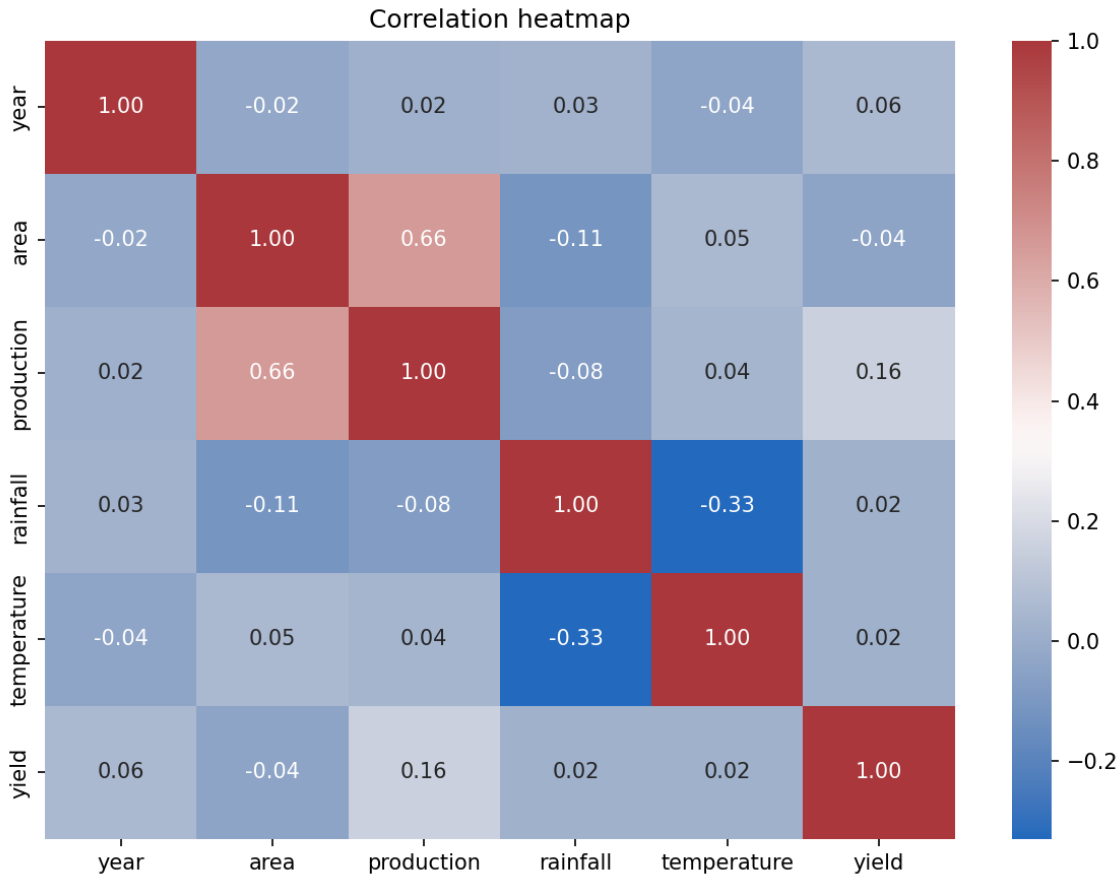
```
yield_vs_rainfall.png
```



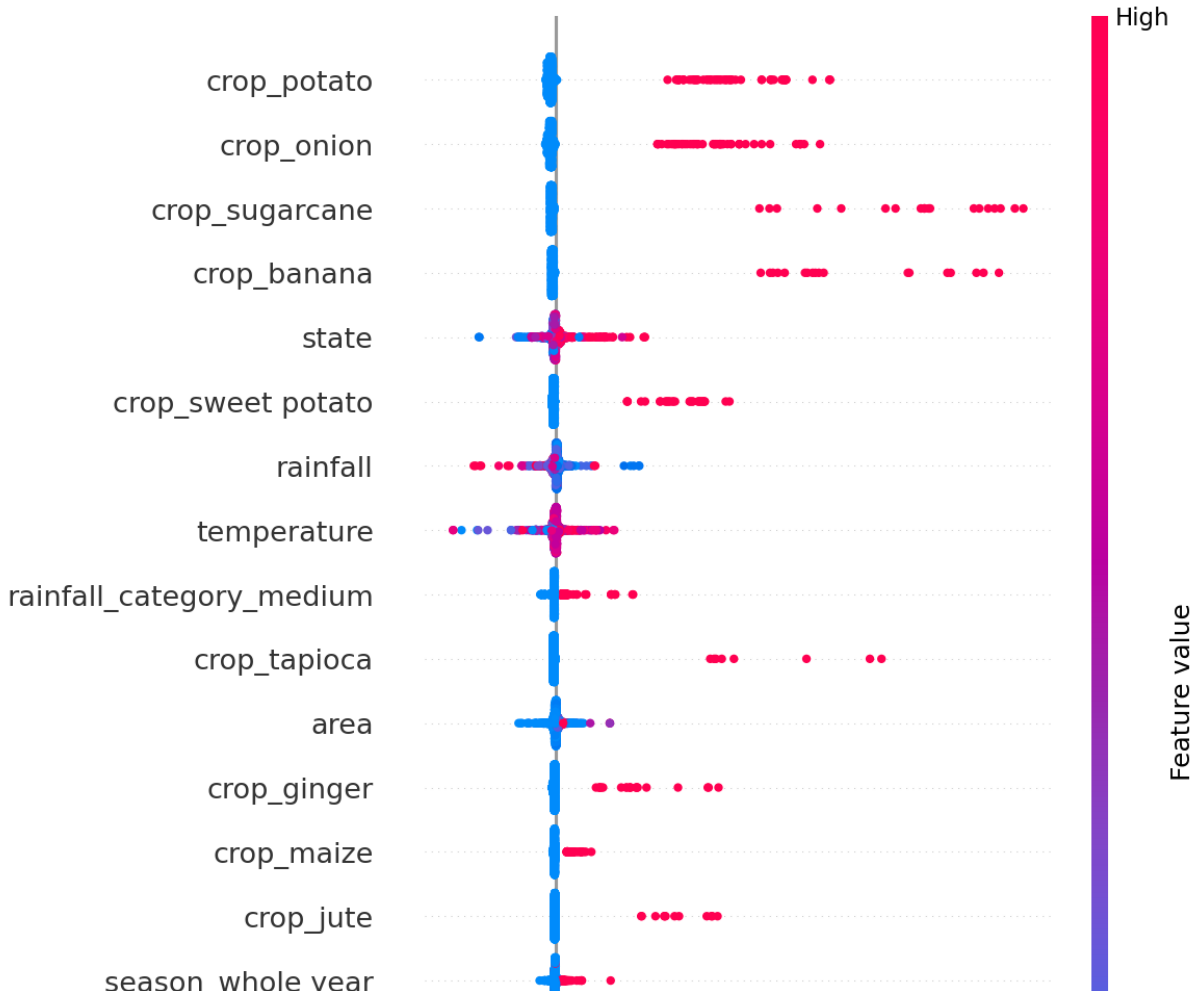

```
crop_distribution.png
```

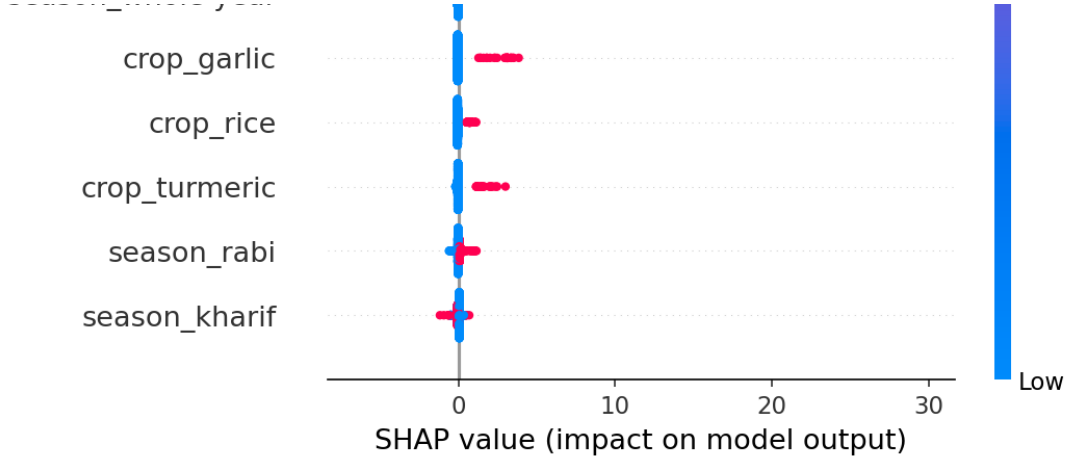



```
correlation_heatmap.png
```



shap_summary_yield.png




crop_garlic
crop_rice
crop_turmeric
season_rabi
season_kharif
0
10
20
30
SHAP value (impact on model output)
Low

9) **Inference** — single prediction with saved pipeline

```
import joblib
import pandas as pd

pipe = joblib.load("ml_models/artifacts/yield_xgb.joblib")

def rainfall_cat(r):
    r = float(r)
    if r < 500: return "low"
    if r <= 1000: return "medium"
    return "high"

examples = [
    {"state": "maharashtra", "crop": "rice", "season": "kharif", "ra
    {"state": "punjab", "crop": "wheat", "season": "rabi", "rainfall
]
for ex in examples:
    ex = dict(ex)
    ex["rainfall_category"] = rainfall_cat(ex["rainfall"])
    row = pd.DataFrame([ex])
    y = float(pipe.predict(row)[0])
    print(ex["state"], ex["crop"], "→ predicted yield (t/ha):", roun
```

```
maharashtra rice → predicted yield (t/ha): 1.442
punjab wheat → predicted yield (t/ha): 3.67
```

Checklist before submitting

- ☐ **Runtime → Run all** completed without errors
- ☐ **File → Download .ipynb** (with outputs)
- ☐ Section **6** shows the metrics **table**
- ☐ Section **7** shows training log + crop accuracy
- ☐ Section **8** shows at least some images

Note: Section 6 and Section 7 both train models (demo table vs full artifact export). For a faster run, increase `DEMO_N` *only in reports or reduce* `AGRIMIND_MAX_TRAIN_ROWS`*.*